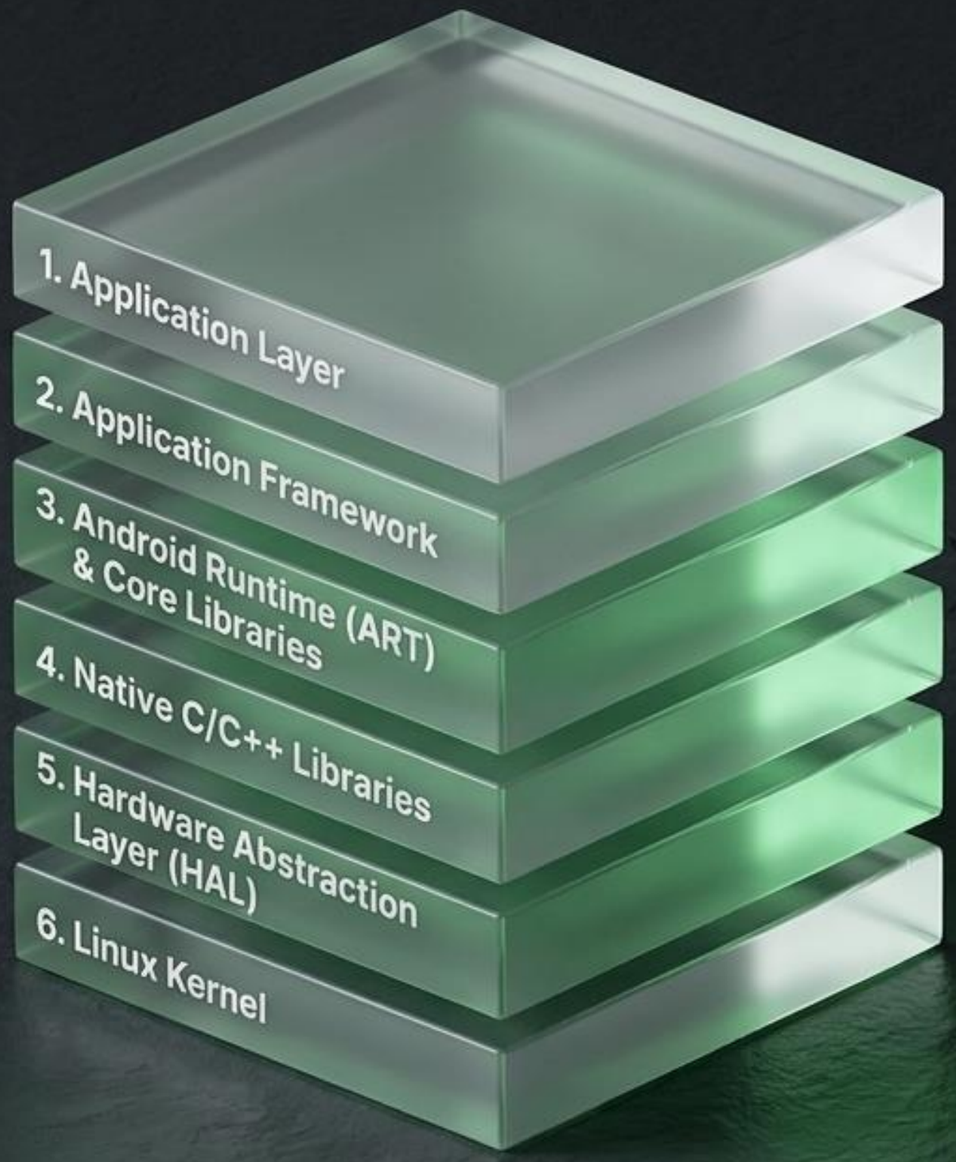
Demystifying Android: The Six-Layer Architecture

From user tap to kernel driver—mapping the internal data flows of the world's most ubiquitous OS.



Architecture

The theoretical blueprint defining how system services, apps, and IPC mechanisms interact.

AOSP (Codebase)

The physical repository where this blueprint is written, compiled, and maintained.

Trace Route: Throughout this deck, we will trace a single user action—opening the Camera App—as it descends through these six layers.

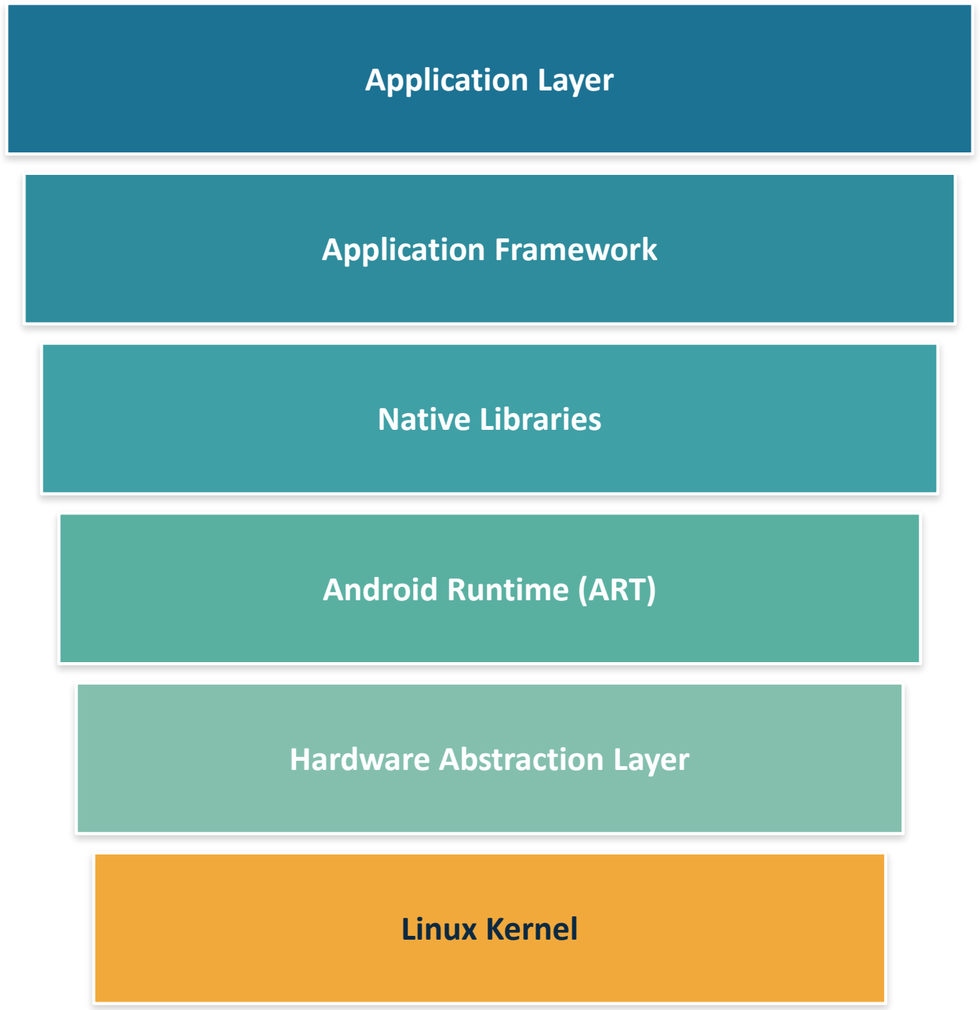
What Is Android Architecture?

A layered OS stack designed to manage hardware, run applications, and provide development tools.

It acts as a bridge between the physical device and applications — organized into five to six key layers, with the Application Framework providing the core APIs developers use to build apps.

AOSP vs. Android Architecture

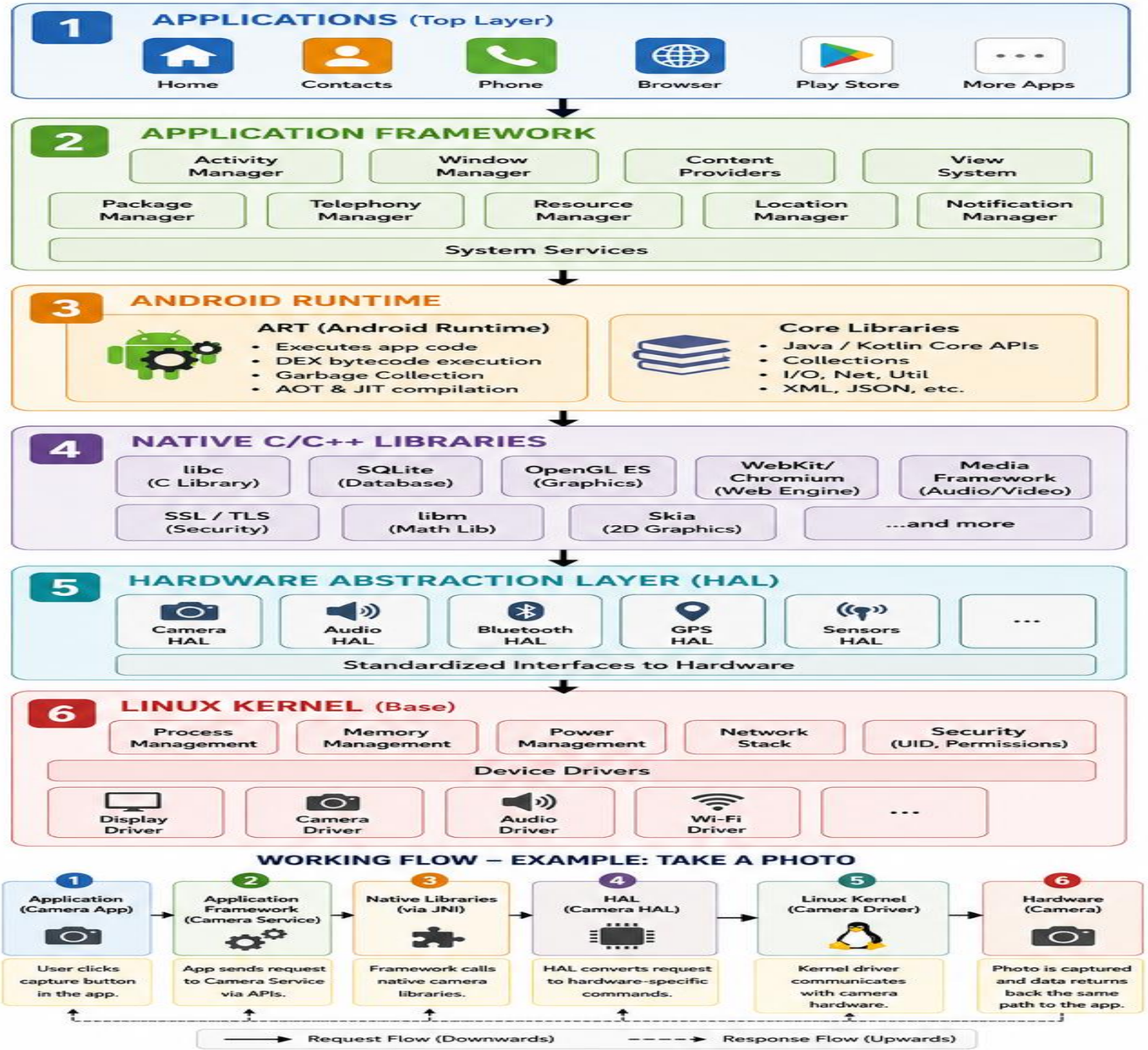
AOSP (Android Open Source Project) is the source code that brings the framework to life. The architecture is the theoretical blueprint defining how system services, apps, and IPC mechanisms interact — AOSP is the physical repository where this entire system is written, compiled, and maintained.



Six layers, top to bottom

ANDROID ARCHITECTURE — LAYERS, COMPONENTS & FLOW

Android is a multi-layered software stack built on Linux kernel. Each layer provides specific services and communicates with adjacent layers through well-defined interfaces.



WORKING FLOW — EXAMPLE: TAKE A PHOTO

LAYERS — DETAILS

1 APPLICATIONS

- Topmost layer containing all user apps and system apps.
- Examples: Dialer, SMS, Camera, Browser, Settings, Play Store, Games, etc.
- Uses services provided by Application Framework.

2 APPLICATION FRAMEWORK

- Provides high-level system services to apps.
- All services are exposed via Java/Kotlin APIs.
- Key Services:
  - Activity Manager: Manages app lifecycle (start, pause, stop, resume).
  - Window Manager: Manages windows & UI.
  - Content Providers: Data sharing between apps.
  - Resource Manager: Access to resources like strings, layouts, images.
  - Notification Manager: Status bar, alerts.
  - Package Manager: App installation, permissions.

3 ANDROID RUNTIME (ART)

- Runtime environment to execute Android apps.
- ART is default runtime (replaces older Dalvik).
- Core Libraries provide essential Java/Kotlin functionality.
- Handles memory management & garbage collection for apps.

4 NATIVE C/C++ LIBRARIES

- Collection of performance-critical libraries written in C/C++.
- Used by framework services through JNI.
- Provides features like database, graphics, media, security, web rendering, etc.

5 HARDWARE ABSTRACTION LAYER (HAL)

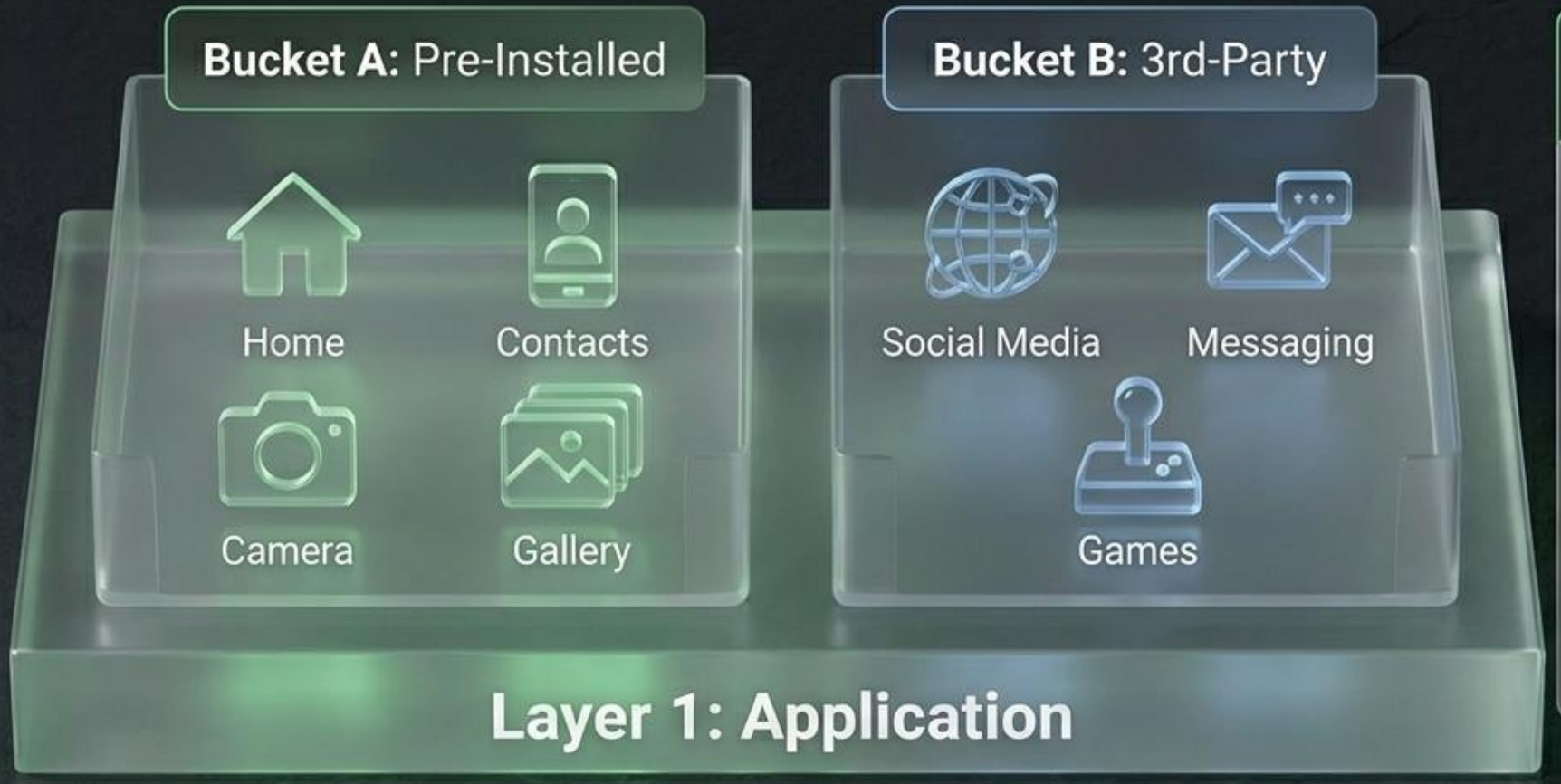
- Provides a standard interface for hardware modules.
- Hides hardware implementation details from upper layers.
- If hardware changes, only HAL needs to be updated, not upper layers.

6 LINUX KERNEL

- Core foundation of Android OS.
- Manages all system resources and hardware.
- Key responsibilities:
  - Process & memory management
  - Power management
  - Network stack
  - Security & permissions
  - Device drivers for all hardware

KEY TAKEAWAYS

- Android uses a layered architecture for modularity and security.
- Apps never directly access hardware.
- Each layer communicates with adjacent layers via defined interfaces.
- HAL makes Android portable across different hardware.
- Linux kernel is the foundation that handles hardware and core system services.

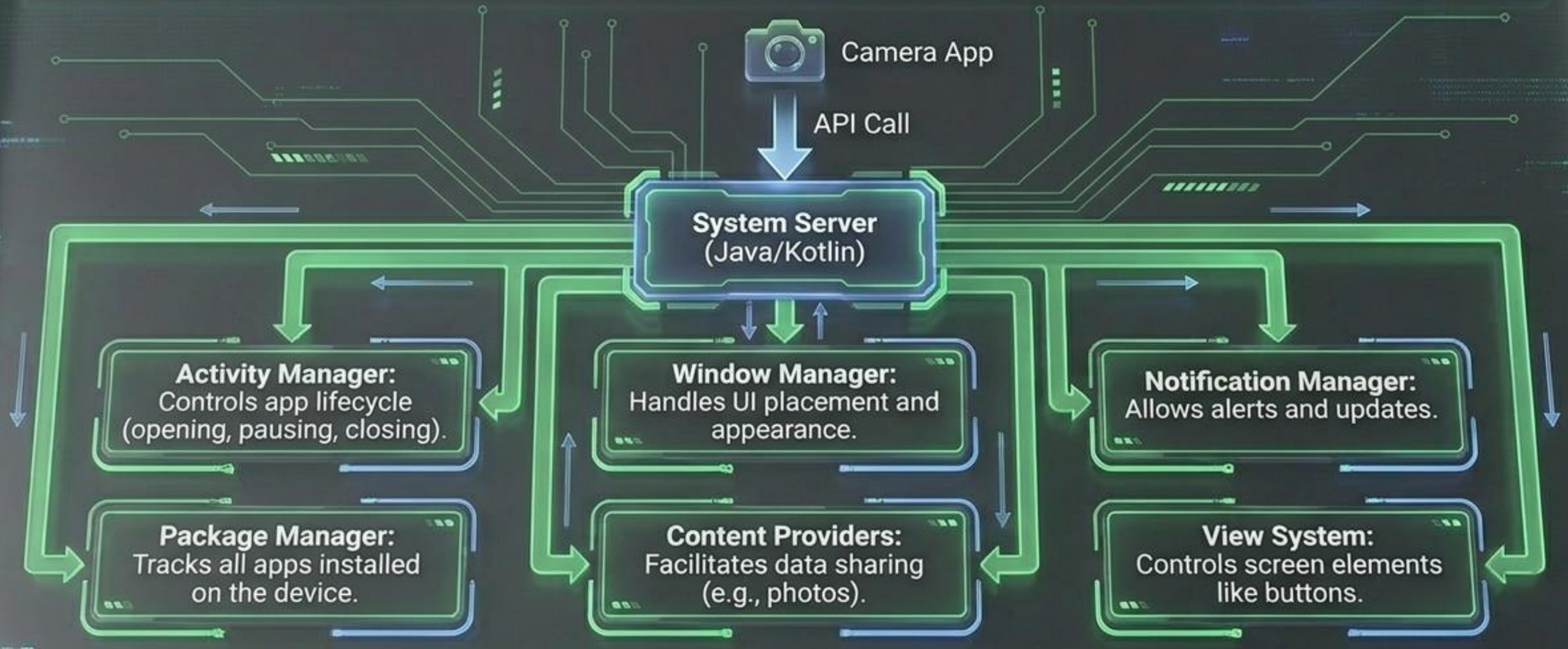


Core Concept

Apps installed here rely entirely on the classes and services provided by the underlying application framework to run at execution time.

Action Initiated: User taps the Camera App icon. The application layer registers the input and prepares to call down to the OS.

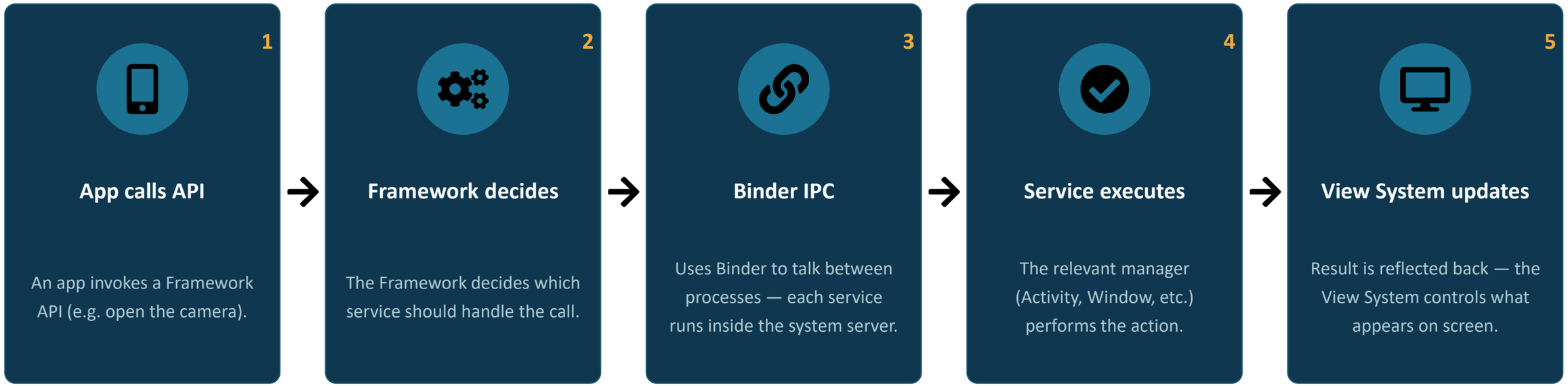
Layer 2: Application Framework



Action Update: The Activity Manager intercepts the Camera tap, waking the app up and managing its lifecycle state.

HOW A USER ACTION TRAVELS THROUGH THE STACK

From Tap to Screen: The Framework Flow



Think of this layer as a controller: a request comes in, the Framework decides which service to use, and that service is like a manager directing the response.

HOW ANDROID PROCESSES TALK TO EACH OTHER

Binder IPC: The Bridge Between Apps

Binder is a system that lets one app talk to another app or system service. In Android, each app runs in a separate process and memory is isolated (sandboxed) — an app cannot directly access another app's or system service's memory.

It works like a client–server model:

Client → requests services

Server → provides services

Binder → acts as the bridge between them

COMPONENTS OF BINDER

1. **Client**

   The app that requests something. e.g. Camera app.

2. **Server**

   A system service or another app that provides the response. e.g. Camera service.

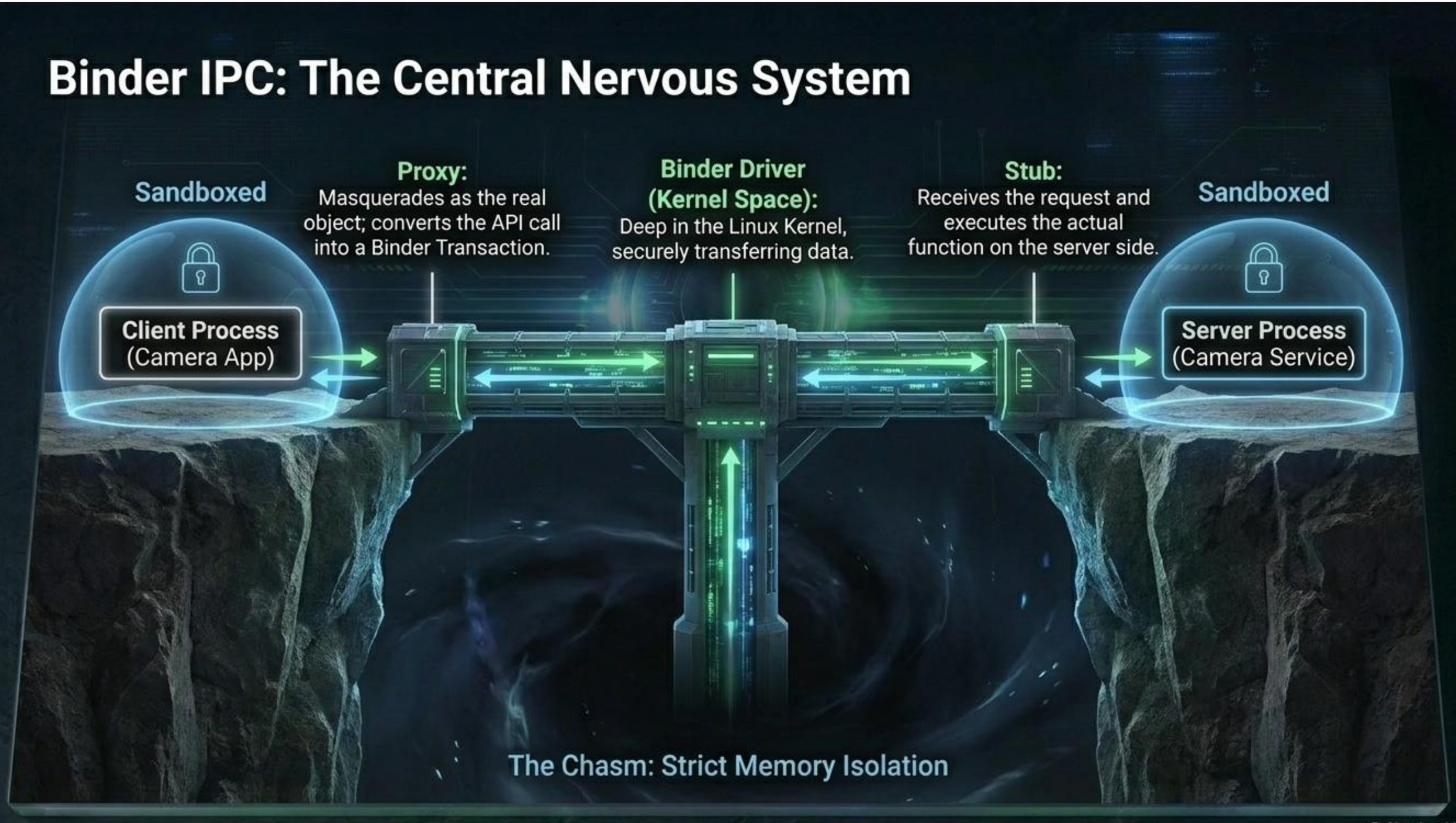
3. **Binder Driver (kernel)**

   The core of Binder IPC — handles communication between processes.

4. **Stub & Proxy**

   Proxy (client side) looks like the real object and forwards requests. Stub (server side) receives requests and calls the actual function.

Binder IPC: The Central Nervous System



Binder in Action: Calling the Camera

1

App (Client)

Calls the Camera API.

2

Proxy object (client side)

Converts the call into a Binder transaction.

3

Binder driver (kernel)

Transfers data securely to the server process.

4

Stub (server side)

Receives the request.

5

Camera service executes

Performs the requested operation.

6

Result returns

The result goes back along the same path.

Android Runtime (ART)

ART contains core libraries and components like the Dalvik VM (DVM). It powers the Application Framework with the help of core libraries — like Java, DVM is a register-based VM specially designed for Android to ensure a device can run multiple instances efficiently. It depends on the Linux kernel layer for threading and low-level memory management.

After Android 5.0, DVM was replaced by ART.

AOT (Ahead-of-Time): compiles app code at install time.

JIT (Just-in-Time):

optimizes further at runtime.

Handles garbage collection (memory cleanup), and each app runs in its own process with its own runtime instance.

Think of it as...

A mini-computer inside the phone that runs your apps.

- **Lightweight**
  Designed specifically for mobile constraints.
- **Efficient memory usage**
  Optimized so multiple apps can run smoothly.
- **Per-app sandboxing**
  Each app runs its own VM for security.
- **Executes .dex files**
  Runs Dalvik Executable bytecode, not raw Java class files.

Layer 3: The Execution Engine & Core Libraries

	Dalvik Virtual Machine (DVM)	Android Runtime (ART)
Era & Footprint	Older engine (Pre-Android 5.0). Register-based, optimized for mobile memory constraints.	Modern engine. Replaced DVM. Manages robust Garbage Collection (memory cleanup).
Compilation Model	Relied heavily on Just-In-Time (JIT) compilation (compiling code continuously at runtime).	Introduces Ahead-Of-Time (AOT) compilation (compiles at install time) paired with optimized JIT.
Key Feature	Lightweight, designed for mobile. Each app runs in its own VM.	App is compiled into native machine code for faster execution.

The DEX Payload

Combines class files, optimizes memory, enables faster execution.

Layer 4: Native C/C++ Libraries

Java/Kotlin Land (Application Framework)

JNI (Java Native Interface)

Native C/C++ Land (Bare-Metal Performance)

Media
Plays and records audio/video formats.

Surface Manager
Manages access to the display subsystem.

SGL & OpenGL
Cross-language APIs for complex graphics processing.

SQLite
Essential lightweight database support.

SSL
Establishes secure, encrypted links.

Action Update: The Camera app pushes raw image data through the JNI to the Surface Manager and OpenGL to render the live, high-framerate viewfinder on screen.

The Zygote Process: Cellular Mitosis for Instant Launches

1. The Template
The "Zygote" process starts when Android boots, acting as a ready-made biological template.

2. Pre-loading
Zygote pre-loads all basic system resources and Core Libraries directly into memory.

3. The Wait
It sits completely idle, listening for an application launch request.

4. The Fork
Upon user tap, Zygote instantly forks (creates a copy of itself) to spawn a new, dedicated process for the app.

Key Insight: Bypassing the need to load core libraries from scratch ensures apps launch almost instantaneously.

Layer 5: Hardware Abstraction Layer (HAL)

Hardware-Facing: Translates standard commands into highly specific electrical instructions.

Software-Facing: Provides a single, uniform standard interface. The same API works for all devices.

Common Modules

- Camera HAL
- Audio HAL
- Bluetooth HAL
- Sensor HAL
- Wi-Fi HAL

System Log
Action Update: The Camera HAL receives a generic 'capture' command and translates it into the precise electrical instruction required by the specific physical lens.

HAL (HARDWARE ABSTRACTION LAYER) – EXPLAINED WITH EXAMPLE

HAL is a bridge between Android system (software) and hardware. It hides hardware-specific details from upper layers and provides a standard interface to interact with hardware.

EXAMPLE: FLASHLIGHT APP (TURN ON TORCH)

1. Flashlight App (User Action)
User opens the flashlight app and taps ON.

2. Application Framework
App sends request to the system service (Camera/Flash Service).

Camera/Flash Service
(System Service)

3. HAL (Camera HAL)
HAL receives the request and converts it into hardware-specific commands.

Camera HAL
(Standard Interface)
This HAL works with different devices and hardware.

4. Linux Kernel Driver
HAL calls the kernel driver to communicate with hardware.

LED Flash Driver

5. Hardware (LED Flash)
Driver sends electrical signal to LED flash.

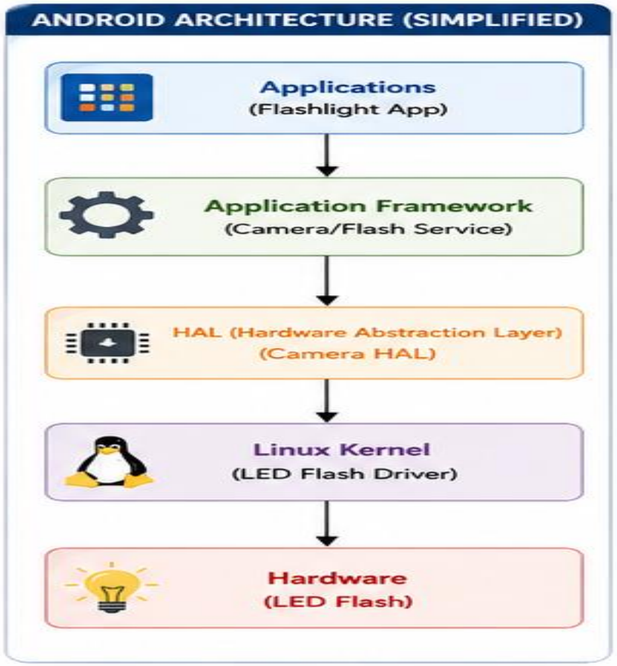
LED Flash turned ON

HOW IT WORKS (STEP BY STEP)

1. User taps ON in the Flashlight App.
2. App Framework (Camera/Flash Service) receives the request.
3. Camera HAL translates the request into hardware-specific command.
4. HAL sends the command to Linux Kernel Driver.
5. Driver controls the LED Flash hardware and turns it ON.

KEY POINT
The app does not need to know how the hardware works. HAL handles all the hardware details.

ANDROID ARCHITECTURE (SIMPLIFIED)



ANOTHER EXAMPLE: MUSIC APP (PLAY SONG THROUGH HEADPHONES)

1. Music App
User presses play in the music app.

2. Application Framework
Audio Service receives the request.

Audio Service

3. HAL (Audio HAL)
Audio HAL processes the request and prepares it for the specific audio hardware.

Audio HAL
(Standard Interface)
Works on all devices

4. Linux Kernel Driver
Audio driver sends digital signal to audic hardware.

Audio Driver

5. Hardware (Headphones)
Signal is converted to sound and you hear the music.

WHY HAL IS IMPORTANT?

- Provides hardware independence
- Same Android OS works on different devices
- Easy to add support for new hardware
- Improves modularity and maintenance

EXAMPLE OF HAL MODULES

Camera HAL | Audio HAL | Bluetooth HAL | Sensors HAL | Wi-Fi HAL | ...and more

SUMMARY FLOW

App (You) → HAL (Translator) → Driver (Controller) → Hardware (Device)

Layer 6: The Linux Kernel

Security Framework

Handles deep memory management to enforce strict isolation and sandbox security.

Process Management

Allocates device resources efficiently exactly when processes demand them.

Network Stack

Handles all low-level network communication and routing.

Driver Model

The heart of the kernel. Manages physical drivers (display, camera, bluetooth). Manufacturers build drivers directly into this layer.

System Log

Action Update: The Camera Driver in the Linux Kernel receives the HAL instruction, physically powering on the camera sensor and opening the aperture.

The Full Descent: From Tap to Hardware

User Space: The App (Client) is tapped by the user.

Framework: The App calls the standard Camera API.

Binder IPC: The Proxy converts the call to a secure Binder Transaction.

Native Libs / ART: Heavy processing is routed through the JNI.

HAL: The Camera Service sends the request to the Camera HAL.

Kernel Space: The Linux Kernel Camera Driver executes the hardware operation.

Physical Device:
The physical shutter clicks.

What feels like an instantaneous tap is actually a complex, multi-layered choreography of sandboxed processes, translated through abstractions, and secured by the kernel.